uranion

Uranion EMEA LTD

uranion.ai - info@uranion.ai

Capitolato per il progetto di Ingegneria del Software

Anno Accademico 2026/2027

FOSSDOT

Smart speaker open source, estensibile e rispettoso della privacy

Proponente: **Uranion EMEA Ltd**

Università degli Studi di Padova, Corso di Ingegneria del Software

Company number: 16356955
Registered office: Portman House Fidcorp, 2 Portman Street, London, W1H 6DU, United Kingdom
Italian branch: Via Tagliamento 14, 00198 Roma — P.IVA IT18154351003 — PEC: uranion.emea@legalmail.it

Uranion EMEA LTD

uranion.ai - info@uranion.ai

Storia del documento

Rev.	Data	Redatto da	Descrizione
0.1	10/09/2026	Uranion	Prima stesura

Uranion EMEA LTD

uranion.ai - info@uranion.ai

Indice

1 Il progetto 4

1.1 Il problema 4

1.2 La soluzione 5

1.3 Sorgenti dell'informazione 7

1.4 Scelte e preferenze tecnologiche 7

1.5 Approfondimenti sulle tecnologie 8

1.6 Criteri di completamento del progetto 10

1.7 Pianificazione delle attività 11

1.8 Licenza 12

2 Riferimenti aziendali e modalità di comunicazione 12

3 Sulla proponente 13

1 Il progetto

1.1 Il problema

Gli smart speaker (Amazon Echo Dot, Google Nest, Apple HomePod) sono ormai oggetti domestici diffusissimi, ma condividono tutti gli stessi limiti di fondo: sono dispositivi chiusi, legati indissolubilmente al cloud del produttore e a un unico assistente vocale. L'utente non può scegliere quale modello di intelligenza artificiale risponda alle proprie domande, non può ispezionare cosa viene inviato ai server remoti, e non ha modo di estendere il dispositivo oltre l'ecosistema di «skill» che il produttore decide di supportare.

A questo si aggiunge un problema di privacy e di sovranità del dato sempre più sentito. Un microfono perennemente in ascolto in salotto o in cucina invia frammenti di audio a un'infrastruttura cloud opaca, sulla quale l'utente non ha alcun controllo. In contesti aziendali, sanitari o pubblici, dove il rispetto del GDPR e la riservatezza delle conversazioni sono requisiti stringenti, questo modello è di fatto inaccettabile, e non esiste un'alternativa self-hosted, verificabile e installabile su un dispositivo fisico.

Vi è infine un tema di durabilità e diritto alla riparazione. Questi dispositivi sono inservibili nel momento in cui il produttore spegne i propri servizi cloud: un hardware perfettamente funzionante diventa rifiuto elettronico per una decisione commerciale unilaterale. Un progetto genuinamente open source, di cui chiunque possa ricostruire hardware, firmware e servizi, elimina alla radice questa forma di obsolescenza programmata.

Il problema che intendiamo affrontare è quindi la mancanza di una piattaforma vocale domestica aperta, estensibile e rispettosa della privacy, che consenta al proprietario di scegliere il provider di intelligenza artificiale, di estenderla tramite protocolli standard e di far cooperare più dispositivi in casa, senza alcun vincolo di lock-in verso un singolo fornitore.

1.2 La soluzione

Immaginiamo un utente attento alla privacy (un maker, ma anche una piccola impresa o un ufficio) che desidera un assistente vocale sul modello dell'Echo Dot, ma di cui possa controllare ogni componente. Installa uno o più dispositivi FossDot in casa o in ufficio; pronunciando una parola di attivazione (*wake word*) il dispositivo cattura la voce, la inoltra a un servizio che fa da ponte verso un provider di intelligenza artificiale a scelta (un LLM in cloud oppure un modello eseguito localmente) e riproduce la risposta sintetizzata. Il proprietario decide quale provider utilizzare, collega servizi esterni tramite *MCP* (Model Context Protocol) e abilita *Skills* e *Agents*. Più dispositivi, registrati sullo stesso account, si riconoscono tra loro e cooperano (annunci in tutta la casa, riproduzione multi-stanza, comunicazione tra stanze), come avviene con una famiglia di dispositivi Echo.

Proponiamo pertanto di realizzare un ecosistema open source completo, rilasciato con licenza permissiva (Apache 2.0 o MIT), che copra l'intera catena del valore di uno smart speaker: dalla scheda elettronica al firmware, dal servizio di back-end all'interfaccia di gestione. Il progetto si articola nelle seguenti quattro componenti.

1. **Selezione dell'hardware.** Individuazione e validazione di una piattaforma hardware di riferimento: scheda di elaborazione (SBC o microcontrollore), array di microfoni (*mic array*) adatto alla cattura vocale a distanza, altoparlante, amplificatore e feedback luminoso. La scelta tiene conto anche del fattore di forma, così che i componenti possano essere racchiusi correttamente in un contenitore adatto a un prodotto finale di questo tipo. L'esito è una distinta base (*bill of materials*) documentata e un prototipo funzionante.
2. **Firmware embedded (MVP).** Sviluppo del firmware che gestisce la cattura audio dall'array di microfoni, il rilevamento della wake word (o l'attivazione manuale), lo streaming dell'audio verso il server, la riproduzione della risposta e il feedback

all'utente. È preferenziale, e costituisce un valore aggiunto, un'implementazione basata su **Rust**.

3. **Servizio server di bridging (MVP).** Sviluppo del servizio di back-end che fa da ponte verso i provider di intelligenza artificiale e verso i servizi di integrazione: pipeline *speech-to-text* → LLM → *text-to-speech*, con un livello di astrazione che rende il provider AI configurabile e sostituibile.
4. **Interfaccia di registrazione dei dispositivi.** Applicazione di gestione multi-dispositivo per registrare i FossDot, associarli a un account, configurarne il provider e i servizi e, sul modello dell'Echo Dot, gestirne l'interconnessione (raggruppamento per stanza, funzioni cross-device).

Sul piano architetturale, il flusso end-to-end attraversa le componenti come riassunto di seguito:

Componente	Ruolo nel flusso
Dispositivo FossDot	Cattura la voce, rileva la wake word, effettua lo streaming dell'audio al server e riproduce la risposta (firmware embedded).
Servizio di bridging	Trascrive l'audio (STT), inoltra la richiesta al provider AI configurato, esegue eventuali Skills/Agents e strumenti MCP, sintetizza la risposta (TTS) e la rinvia al dispositivo.
Provider AI	LLM in cloud o locale, selezionabile dal proprietario tramite il livello di astrazione del servizio.
Servizi MCP	Server esterni (calendario, meteo, domotica, knowledge base) esposti all'assistente come strumenti richiamabili.
Interfaccia di gestione	Registra i dispositivi, ne configura provider e servizi e ne governa l'interconnessione multi-dispositivo.

In sintesi, si propone di realizzare: un ecosistema open source completo (hardware di riferimento, firmware embedded, servizio di bridging e interfaccia di gestione) per uno

smart speaker vocale estensibile, con provider AI configurabili, integrazione di servizi via MCP, Skills e Agents, e supporto multi-dispositivo.

1.3 Sorgenti dell'informazione

Il sistema lavora su sorgenti di natura sia dinamica (i flussi audio) sia statica (configurazioni e registro dei dispositivi), oltre che sulle risposte dei servizi esterni interpellati:

- **Flussi audio dai microfoni**: la voce catturata dall'array di microfoni del dispositivo, reale in esercizio e simulabile in fase di test con campioni audio registrati;
- **Provider di intelligenza artificiale**: le API dei modelli linguistici, in cloud (es. OpenAI, Anthropic) o locali (es. tramite Ollama), che generano le risposte;
- **Servizi di sintesi e riconoscimento vocale** (STT/TTS), come modelli locali o servizi cloud;
- **Server MCP**: servizi esterni integrati come strumenti dell'assistente (calendario, meteo, domotica, basi di conoscenza);
- **Registro dei dispositivi e configurazioni**: identità dei dispositivi, raggruppamenti per stanza, preferenze e credenziali dei provider, generati e gestiti dal sistema stesso.

Uranion non fornisce dataset proprietari: i team utilizzano modelli e servizi open o accessibili pubblicamente e possono simulare l'audio in fase di test. Si segnala che i flussi vocali costituiscono dati personali: il loro trattamento va progettato con attenzione al GDPR, prevedendo, come funzionalità evolutiva, l'opzione di eseguire STT/TTS interamente in locale, così da non trasmettere alcun audio verso il cloud.

1.4 Scelte e preferenze tecnologiche

Uranion non impone uno stack tecnologico: la selezione delle tecnologie spetta al team, che è invitato a individuare e motivare le soluzioni più adatte al caso d'uso. L'unico vincolo **richiesto** è che l'intero progetto sia rilasciato come **open source** con licenza permissiva

(Apache 2.0 o MIT). Ci limitiamo pertanto a una guida di massima sulle aree tecnologiche coinvolte e sulle capacità che ciascuna componente dovrà garantire:

- **Hardware e acquisizione audio:** una piattaforma di elaborazione (single-board computer o microcontrollore, a discrezione del team) e un array di microfoni adeguato alla cattura vocale a distanza; nella scelta dei componenti si tenga conto del fattore di forma, così da poterli racchiudere in un contenitore adatto al prodotto finale;
- **Firmware:** gestione dell'audio, rilevamento della wake word e comunicazione con il server; un'implementazione in **Rust** è considerata preferenziale e costituisce un valore aggiunto, ma non è un vincolo;
- **Servizio di bridging:** integrazione di riconoscimento vocale (STT), modelli linguistici e sintesi vocale (TTS), con un livello di astrazione che renda il provider AI configurabile e sostituibile e un client per il protocollo MCP a supporto dei servizi esterni;
- **Comunicazione tra dispositivo e server:** un canale efficiente e un meccanismo di scoperta dei dispositivi in rete;
- **Interfaccia di gestione:** un'applicazione web per registrazione, configurazione e gestione multi-dispositivo;
- **Delivery:** si incoraggiano la containerizzazione del servizio e l'automazione di build e rilascio.

La scelta degli strumenti concreti per ciascuna area è lasciata al team; nella sezione seguente forniamo alcuni riferimenti come possibile punto di partenza per lo scouting, senza alcun carattere prescrittivo.

1.5 Approfondimenti sulle tecnologie

Uranion mette a disposizione sessioni di supporto tecnico e architetturale. Di seguito alcuni riferimenti utili per lo scouting tecnologico, con particolare attenzione alle

esperienze open source già esistenti nel campo degli assistenti vocali (Willow, Rhasspy / Home Assistant), da cui il progetto può trarre ispirazione:

Tecnologia	Utilizzo	Riferimenti
Rust embedded	Sviluppo di firmware in Rust su microcontrollore (HAL, `no_std`).	https://docs.rust-embedded.org/book/
Embassy	Framework async in Rust per sistemi embedded.	https://embassy.dev
Raspberry Pi	Single-board computer per un prototipo su Linux.	https://www.raspberrypi.com/documentation/
Seeed ReSpeaker	Array di microfoni per cattura vocale a distanza e beamforming.	https://wiki.seeedstudio.com/ReSpeaker/
openWakeWord	Rilevamento della parola di attivazione, open source.	https://github.com/dscripka/openWakeWord
Silero VAD	Voice activity detection leggero e accurato.	https://github.com/snakers4/silero-vad
faster-whisper	Speech-to-text basato su Whisper, ottimizzato.	https://github.com/SYSTRAN/faster-whisper
Piper	Text-to-speech neurale locale e veloce.	https://github.com/rhasspy/piper
Model Context Protocol	Standard aperto per esporre servizi e strumenti agli assistenti AI.	https://modelcontextprotocol.io
Ollama	Esecuzione locale di modelli linguistici (provider AI on-premise).	https://ollama.com
Axum / Tokio	Framework web e runtime asincrono in Rust per il servizio di bridging.	https://github.com/tokio-rs/axum
Willow	Assistente vocale open source, prior art di riferimento.	https://github.com/toverainc/willow

1.6 Criteri di completamento del progetto

La conclusione del progetto è determinata dal raggiungimento dei seguenti obiettivi. Sarà cura del fornitore decidere se fermarsi all'MVP o procedere fino al prodotto finale.

Tutte le componenti applicative devono essere corredate da test concordati con la Proponente; è inoltre richiesto un collaudo end-to-end del sistema nella sua interezza, con la dimostrazione del flusso completo «wake word → domanda → risposta vocale» su un dispositivo fisico.

1.6.1 Consegna minima (MVP)

Realizzazione di un Minimum Viable Product con le seguenti funzionalità:

1. **selezione e validazione dell'hardware di riferimento** (scheda, mic array, altoparlante), con distinta base documentata e prototipo funzionante su scheda di sviluppo, verificando che i componenti scelti siano racchiudibili in un contenitore adatto al prodotto finale;
2. **firmware embedded** capace di catturare l'audio, attivarsi su wake word (o in push-to-talk), inoltrare lo stream al server, riprodurre la risposta e dare feedback visivo;
3. **servizio server** che faccia da ponte verso almeno un provider AI configurabile, con pipeline STT → LLM → TTS e un livello di astrazione predisposto per aggiungere altri provider;
4. **interfaccia di registrazione** di un dispositivo, con associazione a un account e configurazione del provider AI;
5. **flusso end-to-end** dimostrabile su un dispositivo fisico.

1.6.2 Consegna completa

Realizzazione del prodotto finito:

1. tutte le funzionalità dell'MVP;

2. **più provider AI configurabili e commutabili** (cloud e locale, es. Ollama), selezionabili per dispositivo o per utente;
3. **integrazione MCP**: client MCP nel server che espone almeno uno o due servizi esterni (es. calendario, meteo, domotica) come strumenti dell'assistente;
4. **Skills e Agents**: un runtime che consenta di definire Skills (comandi e routine) e Agents (attività multi-step) attivabili vocalmente;
5. **multi-dispositivo e interconnessione**: più dispositivi sullo stesso account, raggruppamento per stanza e funzioni cross-device (annunci whole-home, riproduzione sincronizzata, intercom) sul modello dell'Echo Dot;
6. **firmware in Rust** (se non già realizzato nell'MVP) e/o aggiornamenti OTA;
7. **hardening**: gestione sicura di segreti e chiavi API, autenticazione dispositivo-server e opzione STT/TTS interamente locale per la privacy.

È richiesta inoltre documentazione su scelte implementative e progettuali (con relative motivazioni), problemi aperti e una copertura di test concordata. Per le componenti software (servizio di bridging e interfaccia di gestione) si indica come riferimento una copertura $\geq 80\%$ correlata da report; per il firmware embedded, dove il testing è più oneroso, il livello di copertura sarà concordato caso per caso con la Proponente.

1.7 Pianificazione delle attività

Attraverso un calendario condiviso tra fornitore e Proponente saranno fissate date di revisione dello stato di avanzamento. Gli incontri saranno di tipo:

- **Analisi dei requisiti:** condividere, fissare o aggiornare i requisiti;
- **Formazione:** incontri di formazione tecnica;
- **Stato avanzamento lavori (SAL):** misurare il progresso rispetto alla pianificazione;
- **Dimostrazione:** demo live del prodotto.

Su richiesta del fornitore saranno possibili ulteriori incontri di supporto in base alla disponibilità della Proponente.

1.8 Licenza

Il progetto è concepito come **open source**: il software e i sorgenti hardware prodotti resteranno di proprietà del gruppo, che avrà cura di pubblicarli con licenza permissiva (Apache 2.0 o MIT), coerentemente con la natura del progetto. Il gruppo potrà quindi esporre liberamente il lavoro nel proprio curriculum e contribuire al repository pubblico del progetto.

2 Riferimenti aziendali e modalità di comunicazione

Per il progetto, Uranion mette a disposizione figure di diverso livello per supportare al meglio le esigenze degli studenti. Uranion offre tutto il supporto necessario, inclusi incontri di stato avanzamento lavori (SAL) e sessioni di approfondimento tecnico («tech dive») con i propri esperti e software architect, ogniqualvolta si rendano necessari e ne venga fatta richiesta dai gruppi.

A causa della distanza, le comunicazioni tra i gruppi e i referenti aziendali avverranno principalmente tramite chat (Mattermost self-hosted di Uranion), e-mail e videochiamate (Google Meet). In caso di necessità sarà possibile organizzare incontri in presenza.

Il referente per il progetto è:

Andrea Dorigo

andrea.dorigo@uranion.ai

Si richiede di inserire sempre in copia il referente sopra indicato e info@uranion.ai in ogni comunicazione.

3 Sulla proponente

Nata a Londra, **Uranion EMEA Ltd** mette il proprio know-how tecnologico al servizio delle ambizioni dei propri clienti. Aiutiamo le aziende a trasformare le barriere in opportunità, portando l'AI e la tecnologia dove servono davvero: nei processi quotidiani, nei dati che contano, nelle decisioni strategiche. Con l'AI e tecnologie su misura, ogni progetto diventa un acceleratore di crescita e ogni dato una leva per nuove opportunità.

Uranion opera su progetti di innovazione digitale in Italia e nell'area EMEA, con particolare focus su intelligenza artificiale, integrazione di sistemi, modernizzazione di applicazioni legacy e valorizzazione del patrimonio dati aziendale. Alcuni esempi di progetti realizzati:

- **Migrazione di logiche legacy assistita da AI**: mappatura di regole di business complesse e dipendenze in una knowledge base intelligente e riutilizzabile, trasformando una migrazione rischiosa in un asset di governance dei processi.
- **Piattaforma di supporto decisionale per la gestione fornitori**: storicizzazione e analisi delle performance per rendere le decisioni di approvvigionamento data-driven, rapide e scalabili.
- **Ricerca e raccomandazione basate su AI per cataloghi di prodotto**: ricerca in linguaggio naturale tramite LLM e raccomandazioni visive, per un'esperienza di scoperta personalizzata anche per utenti non tecnici.

Siamo inoltre in prima linea nell'evoluzione del processo di sviluppo software in ottica *agentic coding*: integriamo agenti di intelligenza artificiale lungo l'intero ciclo di vita del prodotto, dall'analisi alla scrittura del codice fino a test e revisione, facendone una leva concreta per accelerare la consegna e innalzare la qualità del software. È una competenza

che mettiamo a disposizione anche dei gruppi, come parte del supporto tecnico al progetto.

Crediamo nel valore del software libero e nella tecnologia open source, che incoraggiamo e a cui partecipiamo attivamente: FossDot ne è un'espressione diretta. Il progetto proposto rappresenta per Uranion un'opportunità per confrontarsi con le nuove generazioni di sviluppatori, facendosi conoscere e creando nuovi legami e network di interesse.